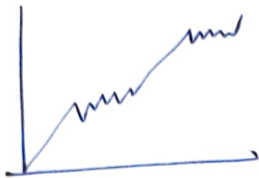


Day 61

Why RNNs are needed?

- 1) Sequence matters in a sentence.
- 2) Time series



→ why?
→ Application
→ Roadmap.

- 3) Speech



Why use RNN?

Input	Output	1st	2nd	3rd	4th
Hi my name is Arifish (5)	0			0	
I love computer (3)	0			0	0
India won the match (4)	1			0	

↓
vectorize

hi → [1, 0, ..., 0] 12 items
my → [0, 1, ..., 1]
...

Zero padding

1st			
2nd			
3rd			

→ Fails in time data
→ 10k nodes input if
large vocabulary.

If max = 1000
min = 5, (5=0) zero padding.

Lot of unnecessary computation.

Problems of zero padding & ANN

- 1) text input $\rightarrow$ varying size
- 2) zero padding $\rightarrow$ unnecessary computation
- 3) Prediction problem
- 4) Losing sequence/hidden meaning. as ANN can't retain memory.

[RNN] $\rightarrow$ [ANN]

Sequence based $\rightarrow$ 1980

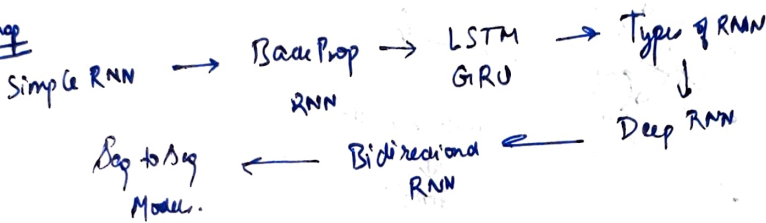
1. Sentiments Analysis

2. Sentence completion.

3. Image Caption Generator.

4. Translating apps.

Roadmap



Lec-56

- 1) Sequence $\rightarrow$ any length,
- 2) Sequence contains some meaning.

ANN $\rightarrow$ fixed input-size

RNN architecture

Data for RNN

(time steps, input-features)

$t=1, t=2, t=3$

Review 1 $\rightarrow$ no. of features

(3, 5)

time steps

Review 2

(3, 5) ; Review 3 (4, 5)

Review	Sentiment
movie was good	1
movie was bad	0
movie was not good	0

Vocab	Sentiment corpus	
movie	was	good
[1 0 0 0 0]	[0 1 0 0 0]	[0 0 1 0 0]
[0 0 0 1 0]	[0 0 0 0 1]	
no bad	not	

Keras $\rightarrow$ Simple RNN $\rightarrow$ batchsize, timestep, input-features

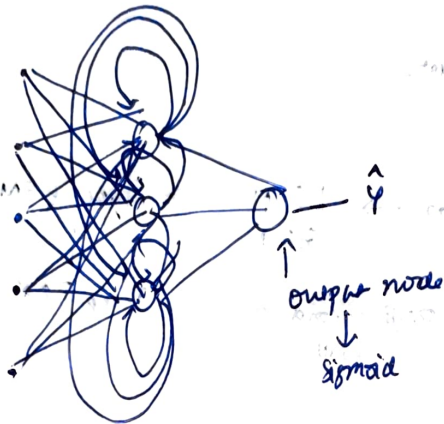
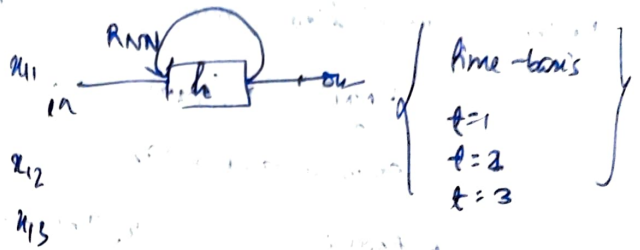
(3, 4, 5)

How RNN works?

X				Sentiment (Y)
x_1	x_{11}	x_{12}	x_{13}	1
x_2	x_{21}	x_{22}	x_{23}	0
x_3	x_{31}	x_{32}	x_{33}	0

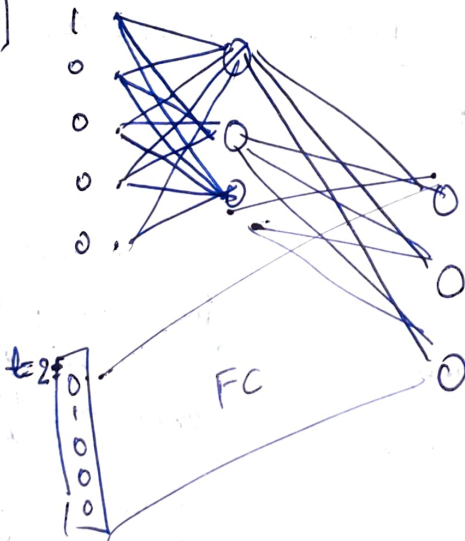
ANN $\rightarrow$ feed forward

hidden



$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

$t=1$



$t=2$

$W(5,3)$
 $W(3,1)$
 $b=3$

$W_1 = 15 \text{ weights}$

$W_2 = 9 \text{ weights}$

$W_3 = 3 \text{ weights}$

$= 27 \text{ weights}$

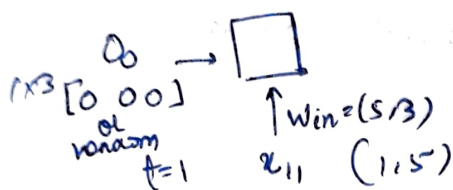
biases = 3 + 1

4 biases

RNN Forward Prop

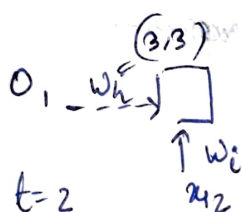
$x_{11} \rightarrow$ vectors 5dim

forward $\rightarrow$ unfolding through time



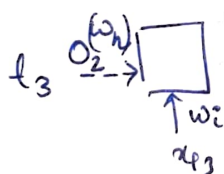
activation = tanh

$$x_{11} \cdot W_i \rightarrow (1, 3) \rightarrow f(x_{11} \cdot W_i + O_0 W_h) \xrightarrow{\text{tanh/recur}} O_1 (1 \times 3)$$



$$f(x_{12} W_i + O_1 W_h) \xrightarrow{\text{tanh}} O_2 (1 \times 3)$$

$$\begin{matrix} x_{12} \cdot W_i & O_1 \cdot W_h \\ (1, 5) \cdot (5, 3) & (1, 3) \cdot (3, 3) \\ \hline 1, 3 & (1, 3) \end{matrix}$$

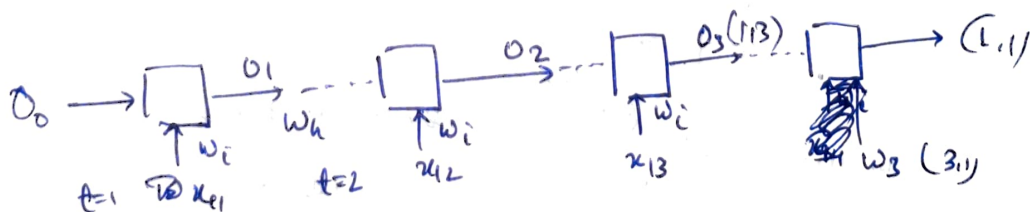


$$f(x_{13} \cdot W_i + O_2 W_h) \xrightarrow{\text{tanh}} O_3 (1, 3)$$

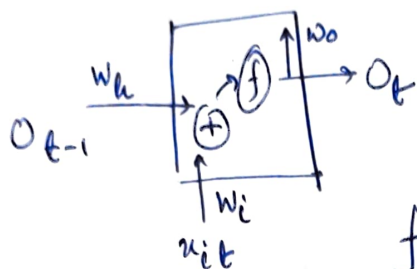
$$O_3 \cdot W_3 \Rightarrow (1, 1) \rightarrow \underline{O_3} \text{ Output}$$

$(1, 3) \cdot (3, 1)$

- 1) Recurrent $\rightarrow$ Hidden layer is recocuring
- 2) New input but same weights
- 3) RNN can process the hidden information behind a sequence.



Simplified Representation



→ tanh

$$f(x_{it} \cdot w_i + O_{t-1} \cdot w_h) \rightarrow \text{vector}$$

i = row #

t = time step

$$f(O_t \cdot w_o)$$

↓ sigmoid/softmax/linear

↑ binary

↑ multi class

↑ regression

Lee-57

RNN Sentiment Analysis

Keras Code Example

↑ integers
encoding

embeddings

text

label

No there
How are you
3 4 5

1

0

→ [1 2 0] ←

→ [3 4 5] ←

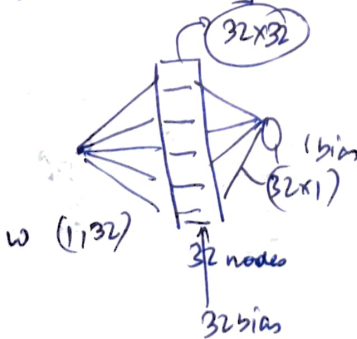
↑
RNN

padding

SD numbers

time step

[25 26 13 19 ...]



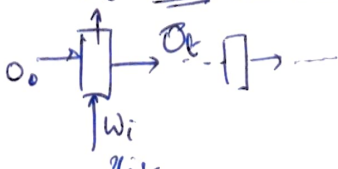
$$\text{nodes: } 32 + 32 + 32 \times 32$$

$$2 \times 1024 + 64 = 1032$$

$$\text{bias} = 33 \text{ bias}$$

$O_t \rightarrow$ shows intermediate results

return-sequences = True



Embedding - In NLP, word embedding is representation of words for text analysis, typically in form of real-valued vector that encodes the meaning of the word such that the words that are closer in the vector space are expected to be similar in meaning.

this is nice

$[0.7 \ 0.1 \ 0.3] \rightarrow$ able to capture semantic meaning.

↑ sparse
0

↓ dense

Word Vec
Glove

NLP embedding

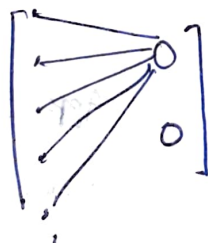
Deep Learning

→ embedding learn

→ Embedding

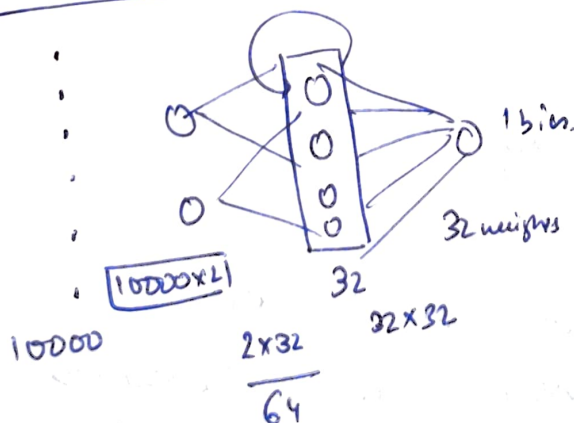
embedding layer ← RNN → sentiment analysis
↓ dense representation

13×2



$[x_1 \ x_2], [x_1$

Model Architecture



Types of RNN

- 1) Many to One
- 2) One to many
- 3) Many to Many

One to One.

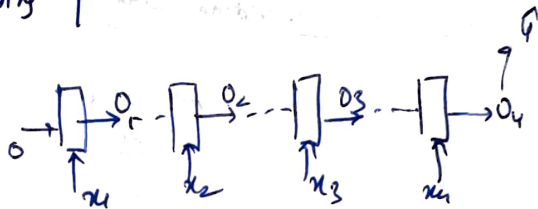
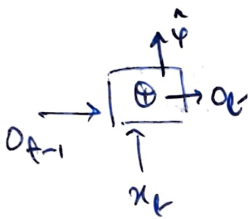
Many to One

same length

variable length.

input $\rightarrow$ sequence $\rightarrow$ Sent, characters, time series
 out $\rightarrow$ non-sequential $\rightarrow$ int/num $\rightarrow$ scalars $-(1,0)$

Sentiment Analysis
 Rating prediction.

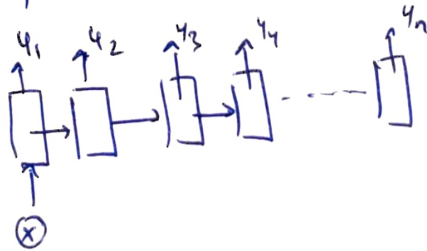


One to many

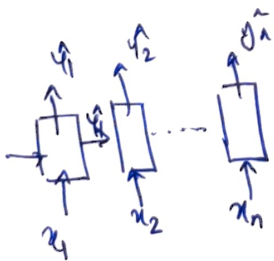
input $\rightarrow$ normal/non-sequential
 output $\rightarrow$ sequential.

Apps:-

1) Image Captioning.



Many to many RNN



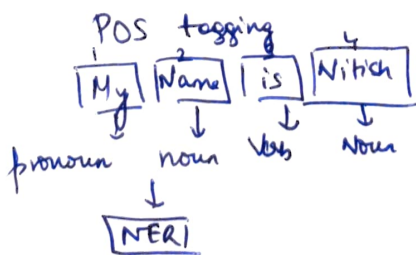
(sequential)
 input $\rightarrow$ Seq-2-seq
 output (sequential)

Same length

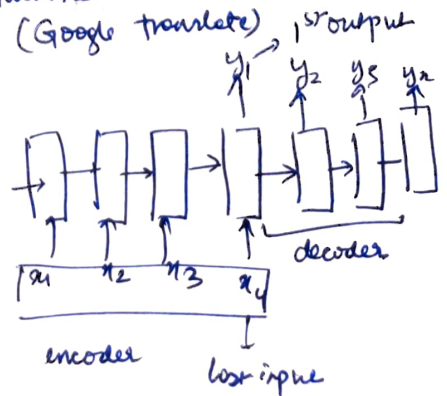
input Seq == output Seq

variable length

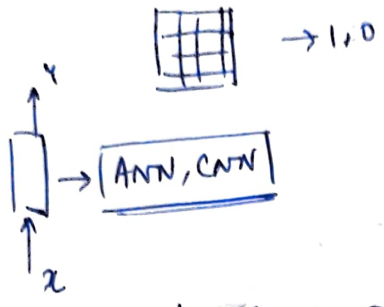
Machine Translation
 (Google translate)



Let's meet at 7pm at the airport



One to one input $\rightarrow$ non-sequential.
 output $\rightarrow$ non-sequential



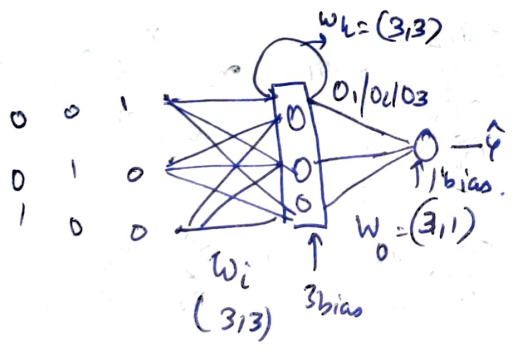
Lec-59 How Backpropagation works in ANN?

Back propagation through time.

text
 cat mat mat 1
 mat mat mat 1
 mat mat cat 0

	x_{11}	x_{12}	x_{13}	
x_1	[1 0 0]	[0 1 0]	[0 0 1]	1
x_2	[0 0 1]	[0 0 1]	[0 1 0]	1
x_3	[0 1 0]	[0 1 0]	[1 0 0]	0

vocab $\rightarrow$ cat Mat Rec
 [1 0 0] [0 1 0] [0 0 1]



$$O_1 = f(x_{11} \cdot w_{i1} + O_0 w_h)$$

$$O_2 = f(x_{12} \cdot w_{i2} + O_1 w_h)$$

$$O_3 = f(x_{13} \cdot w_{i3} + O_2 w_h)$$

$$\hat{y} = g(O_3 \cdot w_o) = \sigma(O_3 w_o)$$

$$L = -y_i \log \hat{y}_i - (1 - y_i) \log (1 - \hat{y}_i)$$

minimize L (Gradient Descent)

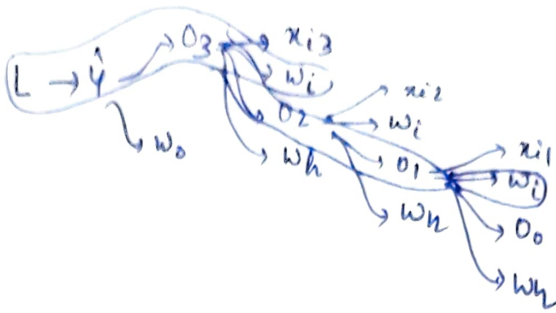
$$w_i = w_i - \eta \frac{\partial L}{\partial w_i} \quad w_h = w_h - \eta \frac{\partial L}{\partial w_h} \quad w_o = w_o - \eta \frac{\partial L}{\partial w_o}$$

$$\frac{\partial L}{\partial w_o} = \frac{\partial}{\partial w_o} \left[-y_i \log (\sigma(O_3 w_o)) - (1 - y_i) \log (1 - \sigma(O_3 w_o)) \right]$$

$$= \left(\frac{-y_i}{\hat{y}_i} + \frac{(1 - y_i)}{(1 - \hat{y}_i)} \right) \times \sigma(O_3 w_o) [1 - \sigma(O_3 w_o)] \times O_3$$

$$= \frac{(-y_i + \hat{y}_i y_i + \hat{y}_i - y_i \hat{y}_i)}{\hat{y}_i (1 - \hat{y}_i)}$$

$$\frac{\partial L}{\partial w_i}$$

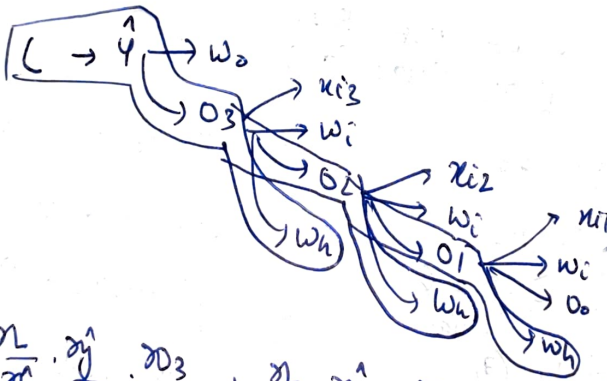


$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i} + \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i} \cdot \frac{\partial w_i}{\partial w_i}$$

$$+ \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i} \cdot \frac{\partial w_i}{\partial w_i} \cdot \frac{\partial w_i}{\partial w_i}$$

$$= \sum_{j=1}^{t-3} \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

$$\frac{\partial L}{\partial w_h}$$



$$\frac{\partial L}{\partial w_h} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_h} + \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_h} \cdot \frac{\partial w_h}{\partial w_h} + \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_h} \cdot \frac{\partial w_h}{\partial w_h} \cdot \frac{\partial w_h}{\partial w_h}$$

$$\frac{\partial L}{\partial w_h} = \sum_{j=1}^n \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_h}$$

Lec-60 Problems with RNN

RNNs → sequential data → textual, time-series

Suffer ② major problems → LSTMs

1. Problem of Long Term dependency
2. Vanishing gradients

→ Marathi is spoken in Maharashtra.
short

→ Maharashtra ————— long sentence

Vanishing Gradient Problem

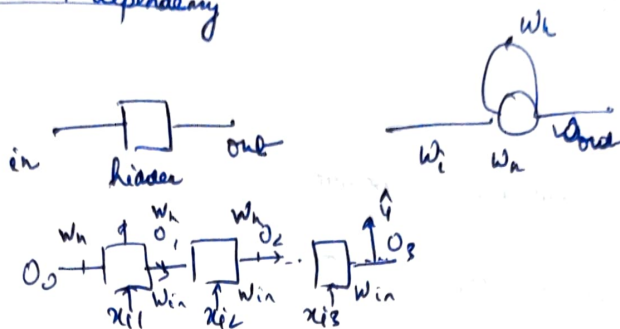
Exploding gradient problem $\rightarrow$ RNN

Possible solutions

Problem #1 $\rightarrow$ Problem of long term dependency

Input	Output
0 0 1	1
0 0 1	0
0 0 0	0
1 1 1	1

3 time steps



$$\frac{\partial L}{\partial w_{in}} = \sum_{j=1}^n \frac{\partial L}{\partial y_j} \cdot \frac{\partial y_j}{\partial o_j} \cdot \frac{\partial o_j}{\partial w_{in}}$$

$$\textcircled{1} \left[\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_{in}} \right] + \textcircled{2} \left[\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_{in}} \right]$$

↓
Short term dependency

$$\textcircled{3} \left[\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_{in}} \right]$$

↓
Long term dependency

$$\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial o_{100}} \times \frac{100}{t=2} \left(\frac{\partial o_t}{\partial o_{t-1}} \right) \frac{\partial o_1}{\partial w_{in}}$$

$$o_1 = \tanh(x_{i1}w_{in} + o_0w_h)$$

$$o_{t+2} = \tanh(x_{it}w_{in} + o_{t-1}w_h)$$

$$\frac{\partial o_t}{\partial o_{t-1}} = \tanh'(\underbrace{x_{it}w_{in} + o_{t-1}w_h}_{0-1}) w_h$$

$$\Rightarrow \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial o_{100}} \times \frac{100}{t=2} \left(\underbrace{\tanh(\quad)}_{0-1} \underbrace{w_h}_{0-1} \right) \frac{\partial o_1}{\partial w_{in}} \quad \left(\begin{array}{l} \text{Vanishing} \\ \text{Gradient} \\ \text{Problem} \end{array} \right)$$

this entire thing will be close to zero.

$\Rightarrow$ All contribution comes from recent inputs.

Solutions &

- ①. Diff. activation $\rightarrow$ rule / leaky rule
- ②. Better weight init
- ③. Skip RNNs
- ④. LSTMs

Problem 2 #Unstable Gradients

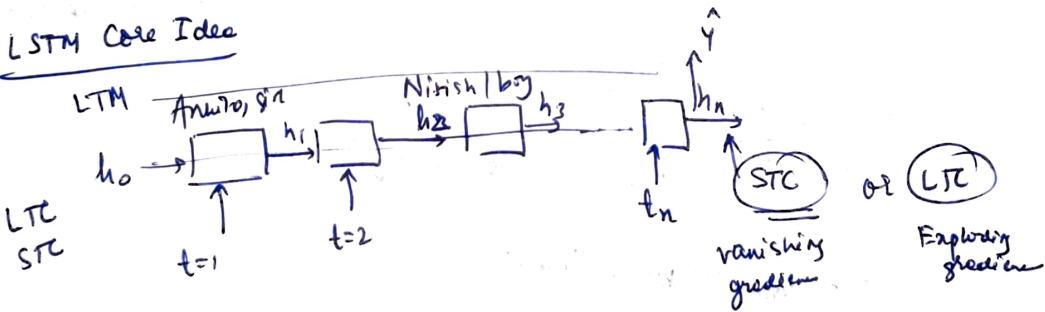
① when wt. init > 1

② $\eta \uparrow$

- 1) Gradient clipping (Keeps in long situations)
- 2) Controlled learning rate
- 3) LSTMs

lec-61 LSTM | Long Short Term Memory

LSTM Core Idea

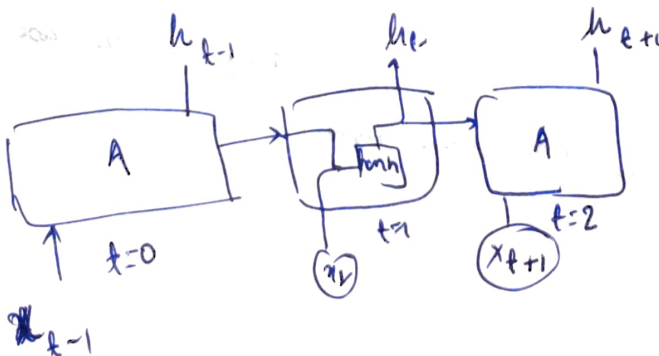
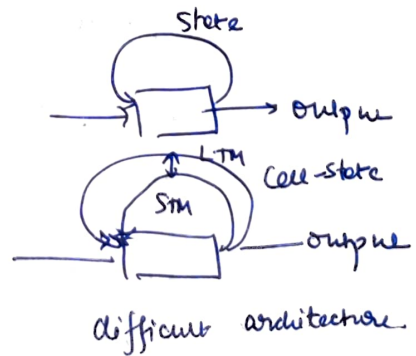


1000 protop
my 2
LTC (Long term context)

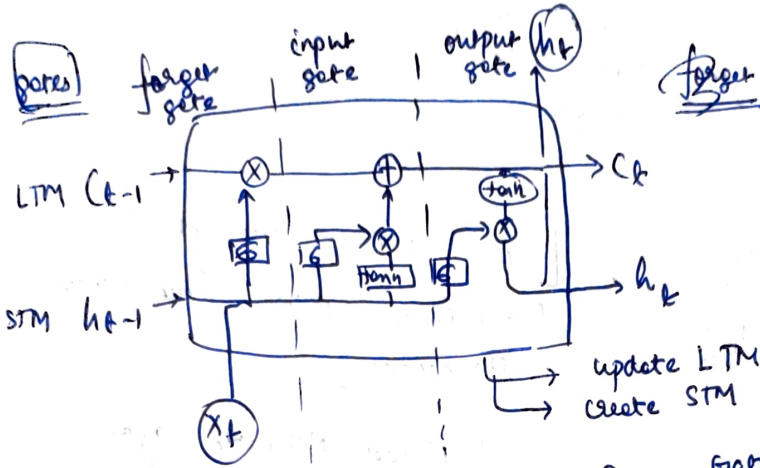
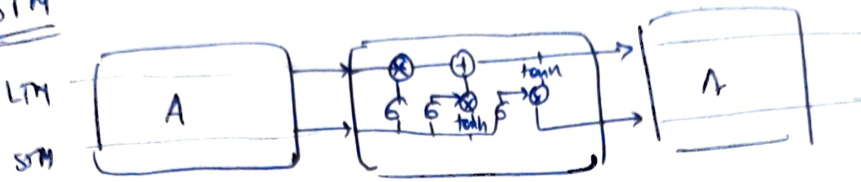
1000 protop
my 2
STC (Short term context)

RNN

LSTM



LSTM



forget gate
(remove)

input
LTM
remove

input gate
(add)

input
LTM
Add

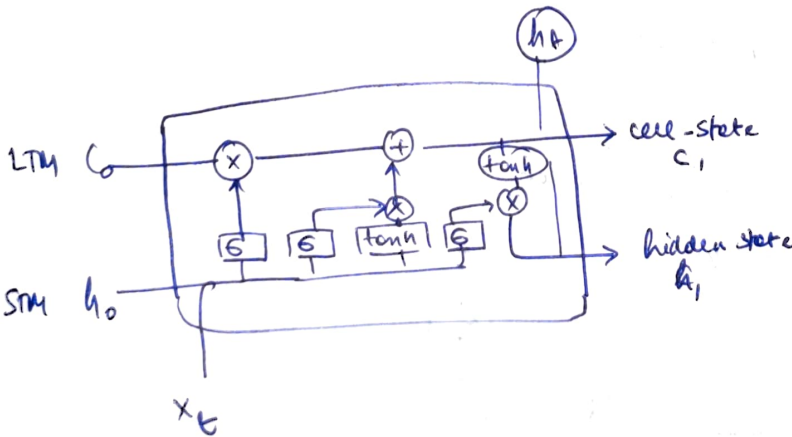
Output Gate
(output)

input
LTM

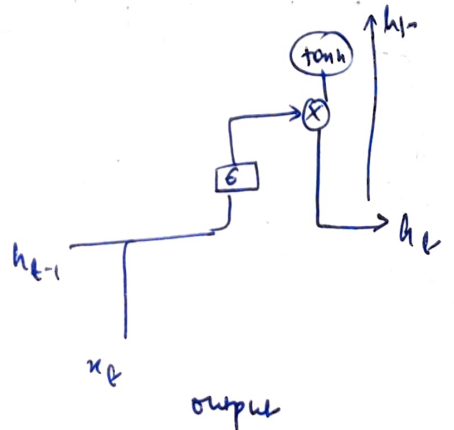
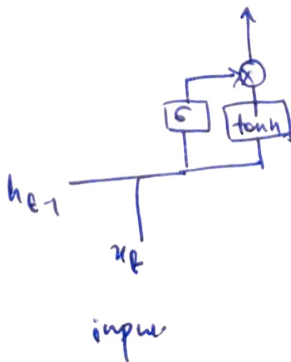
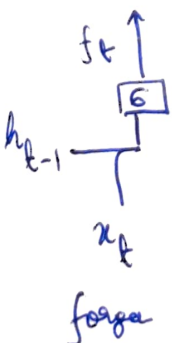
output (Yes/No)

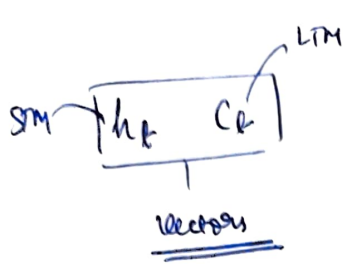
→ Creates STM for intermediate stages.

Lec-32 LSTM Architecture



in $\rightarrow$ $\rightarrow$ output C_t, h_t
 C_0, h_0, x_t
 process
 (Update cell state)
 (calculate h_t)
 3 steps involved.





$[0.1 \quad 0.3 \quad 0.9] \rightarrow \text{dim. equal.}$
3d vector

x_t vector.

cell $[100]$ $t=1$
net $[010]$ $t=2$
net $[001]$ $t=3$

f_t forget gate
 i_t input gate
 c_t candidate cell state
 o_t output gate

$[c_t]$: Candidate cell state
[All these vectors have same dimensions]

Point-wise Operations

$\otimes$, $\oplus$, $\tanh$

$c_{t-1} = [4 \quad 5 \quad 6]$

$f_t = [1 \quad 2 \quad 3]$

$c_{t-1} \otimes f_t \rightarrow \text{vector}$
 $[4 \quad 10 \quad 18]$

$c_{t-1} \oplus f_t \rightarrow [5 \quad 7 \quad 9]$

$\tanh(c_{t-1}) \rightarrow [0.26 \quad 0.34 \quad 0.53]$

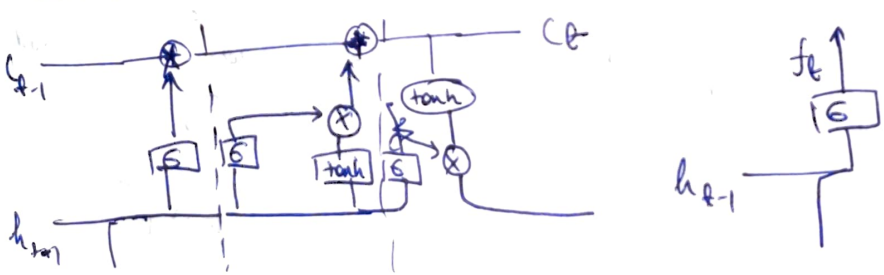
Neural Network layers



no. of units is flexible (hyperparameters)

The forget Gate

input output $\rightarrow$ remove some thing from cell state

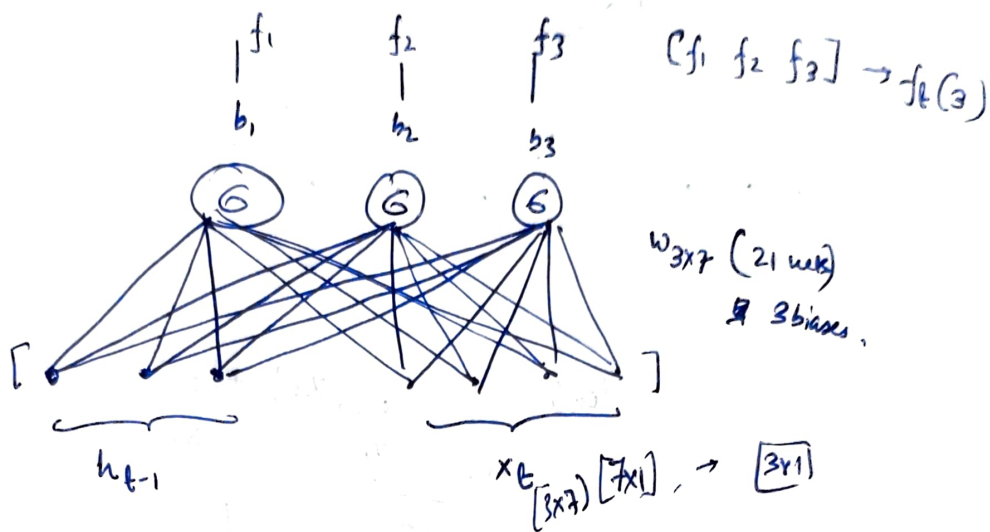


x_t 4dim
 $[x_{t1} \quad x_{t2} \quad x_{t3} \quad x_{t4}]$

$[6]$, num units $\rightarrow 3$

1) Calculate f_t

2) $C_{t-1} \otimes f_t \rightarrow$ remove



$$f_t = \sigma \left(w_f [h_{t-1}, x_t] + b_f \right)$$

3×7 7×1 3×1

$\sigma(3 \times 1) \rightarrow (3 \times 1)$

$$[C_{t-1} \otimes f_t]$$

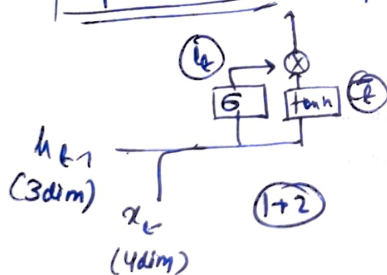
$(C_{t-1}) \rightarrow$ remove.

$$\begin{matrix} \text{---} \otimes \\ (t-1) \uparrow f_t \rightarrow [\frac{1}{2} \ \frac{1}{2} \ \frac{1}{2}] \\ [4 \ 5 \ 6] \end{matrix}$$

(50%, 100%, 60%) $\Rightarrow [2 \ 2.5 \ 3]$

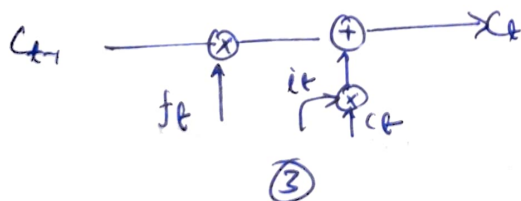
Input Gate

input output $\rightarrow$ add some important info to the C_t



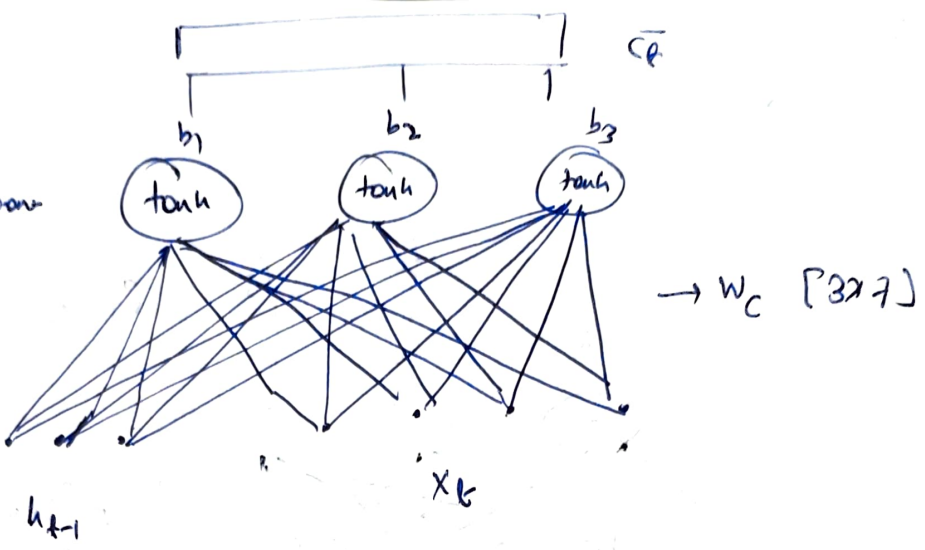
Stages

- 1) $\tilde{C}_t$ calculate (Candidate cell state)
- 2) i_t
- 3) $[C_t] \rightarrow$ cell state.



$\overline{C_t}$

potential important information

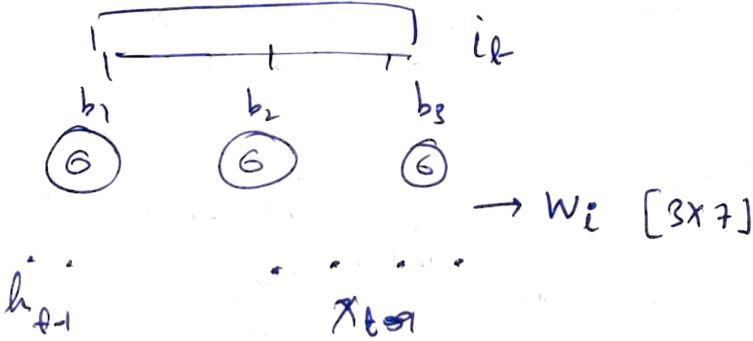


$$\tanh(W_C [h_{t-1}, x_t] + b_C)$$

3×7 7×1 3×1
 $\xrightarrow{3 \times 1}$
 3×1

I_t

filter that decides what info shall go from candidate cell state



$$I_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

Stage 3

point-wise multiplication

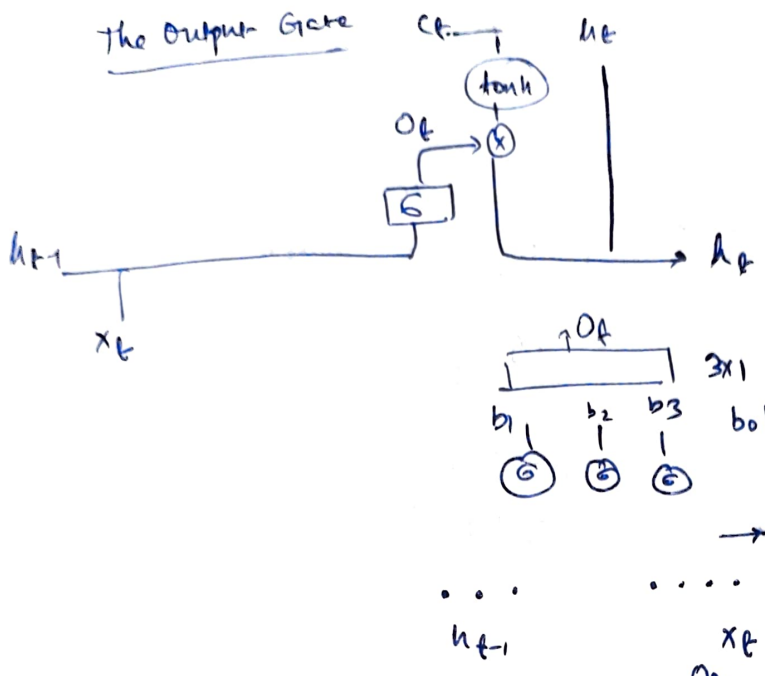
$$I_t \otimes \overline{C_t} \rightarrow \overline{C_t}^f \quad (\text{filtered candidate cell state})$$

$[0.5 \ 0.5 \ 0.5]$ $[4 \ 5 \ 6]$

$$C_t = \overline{C_{t-1}} \otimes I_t + \overline{C_t}^f$$

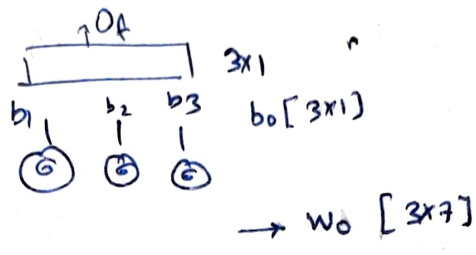
LSTM can do it is more initial info context. to the end of the context

The Output Gate

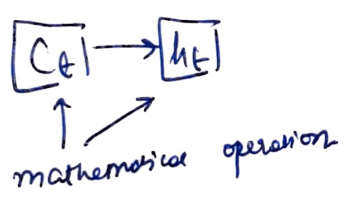


Stages

- ① $\tanh(c_t)$ $[-1, 1]$
 ↑
 filtration
 ↳ we need O_t
- ② $[O_t \otimes \tanh(c_t)]$



$$h_t = \tanh(c_t) \otimes \underbrace{\sigma(w_o \cdot [h_{t-1}, x_t] + b_o)}_{\text{filtration}}$$



$$\begin{aligned} f_t &= \sigma(w_f \cdot [h_{t-1}, x_t] + b_f) \\ \bar{c}_t &= \tanh(w_c \cdot [h_{t-1}, x_t] + b_c) \\ i_t &= \sigma(w_i \cdot [h_{t-1}, x_t] + b_i) \end{aligned}$$

Cell state

$$c_t = c_{t-1} \otimes f_t \oplus i_t \otimes \bar{c}_t$$

$$O_t = \sigma(w_o \cdot [h_{t-1}, x_t] + b_o) \quad ; \quad h_t = \underbrace{O_t + \tanh(c_t)}_{\text{filtration}}$$

Revision

